

GUIA USO DE GIT E GITHUB

● **AVISO CRÍTICO:** Como o laboratório é compartilhado com outra turma, se você pular o checklist de segurança, você vai enviar código na conta de outro aluno ou ter seu projeto completamente bloqueado! Siga rigorosamente os passos abaixo diariamente.

FLUXO 1: CHECKLIST DE SEGURANÇA (Fazer TODA VEZ ao sentar no PC)

1. Apagar as Credenciais Antigas do Windows (Obrigatório)

O Windows salva a senha do último aluno que usou a máquina. Apague-a antes de começar qualquer código:

1. No menu Iniciar, digite e abra o **Gerenciador de Credenciais**.
2. Clique na aba **Credenciais do Windows**.
3. Procure na lista por itens que comecem com **git** ou **GitHub**.
4. Clique neles e selecione **Remover**.

2. Verificar quem está ativo agora

Abra o terminal dentro da pasta do seu projeto e execute os comandos de verificação:

```
git config user.name
git config user.email
```

● **LEMBRETE:** Se o nome ou o e-mail exibidos na tela não forem os seus, você precisará obrigatoriamente rodar os comandos do Fluxo 2 (Configuração de Identidade Local) para não misturar os trabalhos.

FLUXO 2: CONFIGURAÇÃO INICIAL (Executado APENAS UMA VEZ por projeto)

● **LEMBRETE DE FREQUÊNCIA:** Os comandos abaixo servem exclusivamente para criar o projeto localmente e conectá-lo à internet. Você só digita eles uma única vez no primeiro dia do desenvolvimento! Não os repita nas próximas aulas.

```
git init
```

-> Inicializa o repositório local. (Você compra o carrinho de compras vazio).

```
git config --local user.name "Seu Nome"
```

-> Identifica o autor do código localmente. (Coloca o seu nome na ficha de cliente do mercado).

```
git config --local user.email "Seu Email"
```

-> Identifica o contato do autor. (Coloca o seu e-mail na ficha de cliente do mercado).

```
git branch -M main
```

-> Define o nome da ramificação principal. (Determina que a sua rodovia principal se chama "main").

```
git remote add origin "LINK_DO_REPOSITORIO"
```

-> Vincula seu computador ao repositório remoto. (Digita o endereço exato da sua casa no GPS do Uber).

FLUXO 3: ROTINA DIÁRIA (Executado VÁRIAS VEZES enquanto programa)

● **LEMBRETE:** Esta é a sequência padrão que você e sua equipe vão repetir continuamente toda vez que finalizarem uma etapa ou funcionalidade do código.

```
git pull origin main
```

-> Traz as últimas atualizações do GitHub para o seu computador. (Traz novas encomendas da nuvem direto para o seu carrinho antes de você começar a comprar).

● **ATENÇÃO:** Faça sempre o git pull antes de começar a codificar do dia para evitar conflitos de versões caso seu colega de equipe tenha subido alterações recentemente!

```
git add .
```

-> Adiciona os arquivos modificados para a Área de Staging. (Coloca os novos produtos selecionados dentro do seu carrinho de compras).

```
git commit -m "mensagem explicando o que foi feito"
```

-> Registra a alteração localmente. (Passa as compras no caixa e paga. O Git tira uma foto do carrinho e emite o cupom fiscal do código).

```
git push -u origin main
```

-> Envia os commits locais para o servidor do GitHub. (O Uber acelerando na rodovia principal a caminho de casa! Guarda os arquivos na nuvem).

● **LEMBRETE:** Na primeira vez do dia que você rodar o push, o Windows abrirá uma tela flutuante pedindo login. Entre com a sua conta do GitHub para autorizar o envio em segurança.

FLUXO 4: TRABALHANDO COM BRANCHES (Divisão de Tarefas)

Nova Analogia do Projeto: Imagine que a main é o corredor principal do supermercado. Se você precisa testar um código arriscado, criar um design novo ou fazer uma tarefa sem estragar o projeto do seu colega, você entra em um corredor reservado (Branch). Quando tudo estiver testado e perfeito, você volta e junta as compras no corredor principal (Merge).

1. Criar um novo corredor reservado (Branch)

```
git branch nome-da-branch
```

-> Cria uma ramificação nova baseada de onde você está. (Abre uma seção lateral no mercado para trabalhar isoladamente).

2. Mudar para o corredor novo (Checkout)

```
git switch nome-da-branch
```

-> Transfere o seu terminal para a ramificação escolhida. (Você caminha até o corredor reservado. Tudo o que fizer aqui dentro não afeta a rodovia principal).

3. Enviar a Branch nova pela primeira vez para a nuvem

```
git push --set-upstream origin html
```

-> Envia a ramificação criada (ex: branch 'html') para o GitHub criando o vínculo estável.

● **AVISO CRÍTICO:** Sempre que criar uma branch local e for dar o primeiro 'push' nela, use o parâmetro '--set-upstream origin nome-da-branch'. Isso ensina ao Uber a rota fixa entre o seu computador e o servidor do GitHub!

4. Retornar ao corredor principal

```
git checkout main
```

-> Retorna para a trilha de desenvolvimento principal. (Você sai do corredor específico e volta para a avenida 'main').

5. Trazer as alterações do corredor isolado para a principal (Merge)

```
git merge nome-da-branch
```

-> Une o histórico de alterações da branch na ramificação atual. (Você joga as compras da branch para o carrinho da avenida principal de forma integrada).

FLUXO 5: TRABALHANDO EM EQUIPE (Pegando a Branch do Colega)

Você **NÃO** deve criar a branch manualmente do zero no seu PC. Se o seu colega criou uma branch localmente na máquina dele e mandou para o GitHub, *pois isso causará erros de conexão de histórico*.

Para baixar e trabalhar de forma correta na branch que seu colega enviou, use estes 2 passos:

```
git fetch origin
```

-> Atualiza o catálogo local com o que há de novo no GitHub. (Você liga para a central e baixa a lista atualizada de corredores disponíveis na nuvem).

```
git checkout nome-da-branch
```

-> Cria a branch local automaticamente conectada ao servidor. (O Git local percebe que a branch existe no catálogo da nuvem, cria ela no seu PC, faz o vínculo de rota automaticamente e te joga para dentro dela).

FLUXO 6: CHECKLIST DE SAÍDA (TODA VEZ ao ir embora do Laboratório)

● **AVISO CRÍTICO DO LABORATÓRIO:** Não saia do computador sem limpar suas credenciais! Se você esquecer sua conta ativa, qualquer aluno da outra turma poderá fazer alterações, apagar repositórios e enviar commits fingindo ser você.

1. Abra novamente o Gerenciador de Credenciais do Windows.
2. Vá em Credenciais do Windows.
3. Localize o registro criado do git: ou GitHub durante a aula e clique em Remover.